

Que : Client–Server Architecture in the MEAN Stack

The MEAN stack consists of:

- MongoDB – Database
- Express.js – Backend framework
- Angular – Frontend framework
- Node.js – Runtime environment

It follows a client–server architecture, where the frontend (client) and backend (server) are separated and communicate through HTTP requests.

◆ Explanation

1 Client Side (Frontend – Angular)

- Runs in the browser.
- Handles user interface (UI).
- Sends HTTP requests to the server.
- Receives JSON responses and displays data.

2 Server Side (Node.js + Express.js)

- Handles incoming requests.
- Processes business logic.
- Interacts with the database.
- Sends response back to client.

3 Database (MongoDB)

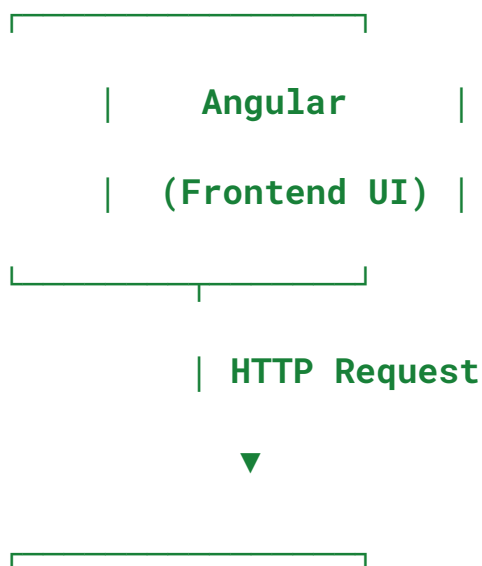
- Stores application data.
 - Server communicates with database to perform CRUD operations.
-

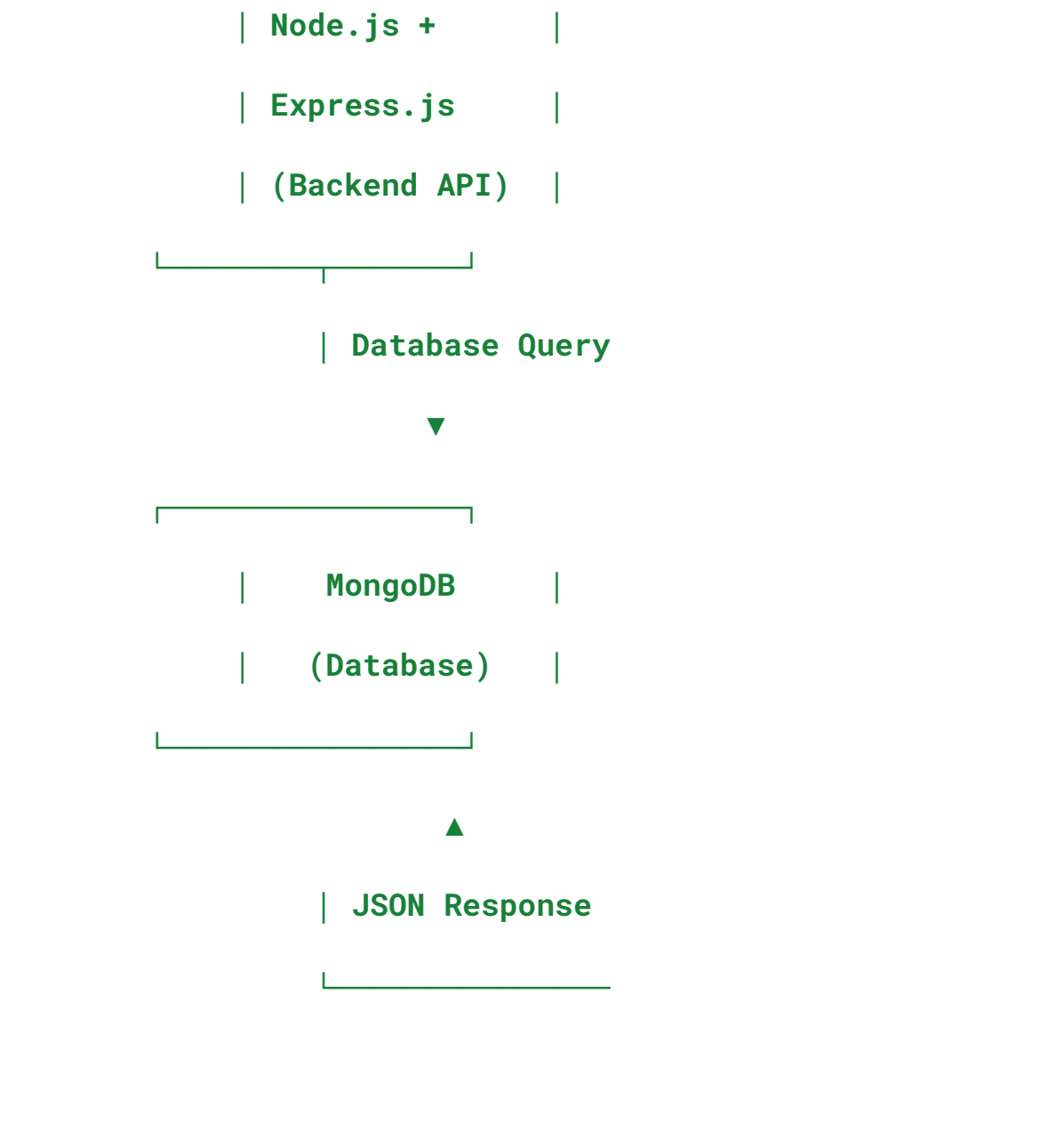
◆ Working Flow

1. User interacts with Angular application.
 2. Angular sends HTTP request to Express server.
 3. Express processes request using Node.js.
 4. Server interacts with MongoDB.
 5. MongoDB returns data.
 6. Server sends JSON response.
 7. Angular displays the result.
-

Diagram of Client–Server Architecture in MEAN Stack

Client (Browser)





✓ Conclusion

In the MEAN stack, Angular acts as the client, Node.js and Express.js act as the server, and MongoDB acts as the database. The client and server communicate using HTTP requests and JSON responses, ensuring a clear separation of concerns and scalability.

✓ What is Node.js and how does it work internally?

Step 1 – Start With Definition (10–15 seconds)

Node.js is a JavaScript runtime environment that allows us to run JavaScript on the server side. It is built on Chrome's V8 engine and is designed for building scalable, high-performance, event-driven applications.

Step 2 – Explain Internal Working (Core Part – 40 seconds)

Internally, Node.js works on a single-threaded event loop architecture. It uses non-blocking I/O, which means it does not wait for operations like file reading or database calls to complete. Instead, it delegates those tasks to libuv, which manages a thread pool for handling asynchronous operations. Once the task completes, the callback is pushed into the event queue, and the event loop executes it.

Step 3 – Explain Why It's Fast (10–15 seconds)

Because Node.js doesn't create a new thread per request and uses an event-driven, non-blocking model, it is highly efficient for I/O-intensive applications like APIs and real-time systems.

Step 4 – Add One Advanced Point (Shows 3-Year Level)

However, since it is single-threaded, CPU-intensive tasks can block the event loop, which is why worker threads or clustering are used in production for scaling.

Final Polished Answer (Say This in Interview)

Node.js is a JavaScript runtime built on the V8 engine that allows JavaScript to run outside the browser. Internally, it follows a single-threaded event loop architecture with non-blocking I/O. When an asynchronous operation like a file read or database call occurs, Node delegates it to libuv's thread pool. Once the operation completes, the callback is placed in the event queue, and the event loop processes it. This makes Node.js highly efficient for I/O-bound and real-time applications. However, for CPU-intensive tasks, we need worker threads or clustering to avoid blocking the event loop.



What is Middleware?

♦ Short Definition (Start Like This)

Middleware is a function in Express.js that executes between the incoming request and the final response. It has access to the request object, response object, and the next function in the application's request-response cycle.

♦ Technical Definition

A middleware function has this signature:

```
(req, res, next) => { }
```

Where:

- `req` → Request object
 - `res` → Response object
 - `next()` → Function to pass control to the next middleware
-

♦ How Middleware Works (Execution Flow)

When a client sends a request:

Client Request



Middleware 1



Middleware 2



Route Handler

↓
Response

Each middleware must call `next()` to pass control forward.
If it does not call `next()`, the request will hang.

◆ Why Middleware is Used?

Middleware is used for:

- Authentication
- Authorization
- Logging
- Error handling
- Data validation
- Parsing JSON
- CORS handling
- Rate limiting

◆ Types of Middleware

① Application-level middleware

```
app.use((req, res, next) => { })
```

② Router-level middleware

③ Built-in middleware

- `express.json()`
- `express.urlencoded()`

④ Third-party middleware

- CORS
- Helmet
- Morgan

5 Error-handling middleware

```
(err, req, res, next) => { }
```

◆ Example

```
app.use((req, res, next) => {  
  console.log("Request received");  
  next();  
});
```

This middleware logs every request before reaching the route.

Best Interview Answer :

Middleware in Express.js is a function that runs between the request and response cycle. It has access to req, res, and next, and it can modify request/response objects, end the response, or pass control to the next middleware. It is commonly used for authentication, logging, validation, error handling, and parsing request data. Middleware execution follows a sequential pipeline model where each middleware must call next() to move to the next function.

Ques : What is REST? What are RESTful principles?

REST stands for **Representational State Transfer**.

It is a way (or style) of designing APIs for communication between a client and a server.

In REST:

- Everything is treated as a **resource** (like users, products, orders).
- Each resource has a **unique URL**.
- We use standard **HTTP methods** to perform actions.

For example:

- **GET** /users → Get all users
- **POST** /users → Create a new user
- **PUT** /users/1 → Update user with ID 1
- **DELETE** /users/1 → Delete user with ID 1

So basically, REST uses URLs + HTTP methods to perform operations.

Main Principles of REST :

REST follows some important rules:

1 Client-Server

The frontend (client) and backend (server) are separate.
They communicate through APIs.

2 Stateless

The server does not remember previous requests.
Each request must contain all the required information (like authentication token).

3 Cacheable

Responses can be cached to improve performance.

4 Uniform Interface

We use:

- Clear URLs for resources
- Proper HTTP methods (GET, POST, PUT, DELETE)
- Standard response formats (usually JSON)

5 Layered System

The client doesn't know whether it is talking directly to the server or through a proxy or load balancer.

6 Code on Demand (Optional)

The server can send executable code (rarely used in APIs).

✓ Final Easy Interview Answer (Very Clear)

REST is a way of designing APIs where everything is treated as a resource and identified by a URL. It uses standard HTTP methods like GET, POST, PUT, and DELETE to perform operations. REST follows principles like stateless communication, client-server separation, cacheability, and uniform interface. Because of these rules, REST APIs are simple, scalable, and widely used in modern backend development.

✓ What is Routing?

Routing means:

Deciding how the server responds to different client requests based on the URL and HTTP method.

In simple words:

Routing tells the server:

- If the request is `/users` → do this
- If the request is `/products` → do that

It connects URLs to specific functions.

Real-Life Example

Think of routing like a **receptionist in a company**:

- If someone asks for Sales → send to Sales department
- If someone asks for Support → send to Support team

Similarly:

- `/login` → Login function
- `/register` → Register function
- `/users` → Get users function

Routing directs the request to the correct handler.

Technical Definition

In Express:

Routing refers to defining application endpoints (URLs) and specifying how the server responds to client requests using HTTP methods like GET, POST, PUT, and DELETE.



Basic Example

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Home Page");
});

app.get("/users", (req, res) => {
  res.send("Users Page");
});

app.post("/users", (req, res) => {
  res.send("User Created");
});

app.listen(3000);
```

Explanation:

- `app.get()` → Handles GET request
- `app.post()` → Handles POST request
- URL + HTTP method together define a route



Route Parameters Example

```
app.get("/users/:id", (req, res) => {
  res.send(`User ID: ${req.params.id}`);
});
```

If request is:

GET /users/10

Output:

User ID: 10

This is called **dynamic routing**.



Types of Routing in Express

- 1 Basic Routing
 - 2 Route Parameters
 - 3 Query Parameters
 - 4 Router-level routing (using `express.Router()`)
 - 5 Middleware-based routing
-



Why Routing is Important

Routing helps:

- Organize application structure
- Separate concerns
- Handle multiple endpoints
- Build REST APIs
- Make application scalable

In large applications, routes are separated into different files like:

- `userRoutes.js`
- `productRoutes.js`
- `authRoutes.js`

This keeps code clean and maintainable.



Best Interview Answer (Professional & Clear)

Routing is the process of defining how an application responds to client requests based on the URL and HTTP method. In Express.js, we define routes using

methods like `app.get()`, `app.post()`, `app.put()`, and `app.delete()`. Each route maps a specific endpoint to a callback function that handles the request and sends a response. Routing helps organize the application, handle multiple endpoints efficiently, and build RESTful APIs in a scalable way.

What is MVC Architecture?

◆ Definition

MVC (Model–View–Controller) is a software architectural pattern that separates an application into three main components: Model, View, and Controller.

It helps organize code in a clean, scalable, and maintainable way.

Components of MVC

1 Model

- Handles data and business logic.
- Interacts with the database.
- Responsible for CRUD operations.

Example:

- User model
- Product model

In Node.js, this usually contains:

- Database schema
 - Query logic
-

2 View

- Responsible for UI (User Interface).
- Displays data to the user.

In backend APIs:

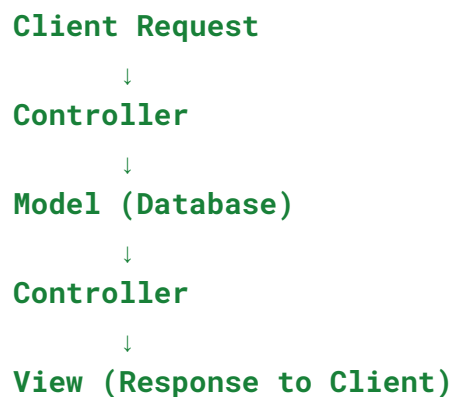
- View is usually JSON response.
 - In templating apps, it can be EJS, Handlebars, etc.
-

3 Controller

- Acts as a bridge between Model and View.
- Handles incoming requests.
- Calls model functions.
- Sends response back to client.

Controller contains application logic.

How MVC Works (Flow)



Step-by-step:

1. Client sends request.
 2. Controller receives it.
 3. Controller calls Model.
 4. Model interacts with database.
 5. Controller sends response (View).
-

Example Structure in Node.js

Folder structure:

project/

```
|
|— models/
|   └─ userModel.js
|
|— controllers/
|   └─ userController.js
|
|— routes/
|   └─ userRoutes.js
|
└─ app.js
```

Example Controller

```
exports.getUsers = (req, res) => {
  const users = userModel.findAll();
  res.json(users);
};
```



Why MVC is Important

- Separation of concerns
 - Clean code structure
 - Easy to maintain
 - Scalable
 - Team-friendly development
-



Best Interview Answer

MVC stands for Model–View–Controller. It is an architectural pattern that separates an application into three components: Model, which handles data and business logic; View, which is responsible for presenting data to the user; and Controller, which handles incoming requests and coordinates between Model and View. MVC improves code organization, scalability, and maintainability, and is commonly used in backend frameworks like Express.js.

1 What is npm?

◆ Definition

npm (Node Package Manager) is the default package manager for Node.js used to install, manage, and share JavaScript libraries and dependencies.

◆ What Does npm Do?

- Installs packages (`npm install express`)
 - Manages dependencies
 - Maintains versioning
 - Runs scripts
 - Publishes packages
-

◆ How It Works Internally

- npm connects to the npm registry.
 - Downloads package + its dependencies.
 - Stores them inside `node_modules`.
 - Updates `package.json` and `package-lock.json`.
-

◆ Example

```
npm install express
```

This:

- Downloads Express
 - Installs dependencies
 - Updates project files
-

2 What is package.json?

◆ Definition

package.json is a metadata file that describes a Node.js project and manages its dependencies.

It is the **main configuration file** of a Node project.

♦ Why It's Important

- Defines project structure
- Tracks dependencies
- Allows others to install project using `npm install`
- Enables script execution

3 What is package-lock.json?

♦ Definition

package-lock.json locks the exact version of installed dependencies and their sub-dependencies.

It ensures **consistent installation across environments**.

♦ Why It Exists

In `package.json`:

```
"express": "^4.18.2"
```

The `^` means:

Install compatible minor/patch updates.

Without lock file:

- Different developers may install slightly different versions.

With `package-lock.json`:

- Exact version numbers are stored.
- Same dependency tree is installed everywhere.

♦ What It Contains

- Exact version
 - Dependency tree
 - Integrity hash
 - Resolved source URL
-

Difference Between package.json and package-lock.json

package.json	package-lock.json
Defines project metadata	Locks exact versions
Editable by developer	Auto-generated
Uses version ranges	Uses exact versions
Controls what to install	Controls how exactly it installs

Best Interview Answer (Professional & Clear)

npm is the default package manager for Node.js used to install and manage dependencies.

package.json is the main configuration file of a Node project that contains metadata, scripts, and dependency information.

package-lock.json is an auto-generated file that locks the exact versions of dependencies to ensure consistent installation across different environments.



Complete Production-Level Project Structure

```
my-app/
├── src/
│   ├── config/
│   │   ├── db.js
│   │   └── env.js
│   ├── controllers/
│   │   ├── user.controller.js
│   │   └── auth.controller.js
│   ├── models/
│   │   ├── user.model.js
│   │   └── product.model.js
│   ├── routes/
│   │   ├── user.routes.js
│   │   └── auth.routes.js
│   ├── middleware/
│   │   ├── auth.middleware.js
│   │   ├── error.middleware.js
│   │   └── logger.middleware.js
│   ├── app.js
│   └── index.js
├── tests/
├── .env
├── .gitignore
├── package.json
├── package-lock.json
└── README.md
```



Explanation Folder by Folder



1 src/

Main source code lives here.

Keeps root clean and production-ready.



2 config/

Contains configuration files.

db.js

- Database connection logic.

env.js

- Environment variable handling.

Why separate?

👉 Keeps configuration isolated from business logic.



3 controllers/

Handles:

- Request
- Response
- Calls service layer

Example:

```
exports.getUsers = async (req, res) => {  
  const users = await userService.getAllUsers();  
  res.json(users);  
};
```

Controller should NOT contain heavy business logic.

4 models/

Contains:

- Database schemas
- ORM/ODM definitions
- Database interaction logic

Example:

- User schema
- Product schema

This is the data layer.

5 routes/

Defines API endpoints.

Example:

```
router.get("/", userController.getUsers);
```

Routes connect:

URL → Controller

6 middleware/

Contains reusable middleware:

- Authentication
- Authorization
- Logging
- Global error handler

Example:

```
app.use(errorMiddleware);
```

Middleware runs before controllers.

app.js

Responsible for:

- Initializing Express
- Applying middleware
- Loading routes

Example:

```
const express = require("express");
const app = express();

app.use(express.json());
app.use("/api/users", userRoutes);

module.exports = app;
```



index.js (or server.js)

Entry point.

Starts server:

```
const app = require("./app");

app.listen(3000, () => {
  console.log("Server started");
});
```



.env

Stores environment variables:

```
PORT=3000
DB_URL=mongodb://localhost:27017/myapp
JWT_SECRET=secret
```

Never commit this file.



How Request Flows in This Structure

Client Request



Route



Middleware



Controller



Service



Model (Database)



Response

This shows clear separation of responsibilities.

Best Interview Answer

In a production-level Express application, we structure the project into folders like controllers, models, routes, middleware, services, config, and utils inside a src directory. Routes define endpoints, controllers handle requests and responses, services contain business logic, models interact with the database, middleware handles cross-cutting concerns like authentication and error handling, and config manages environment and database settings. This structure ensures scalability, maintainability, and separation of concerns.

Ques : HTTP Methods and Their Meanings

HTTP methods are used by clients to perform operations on resources available on a server.

The main HTTP methods are:

1. **GET** – Used to retrieve data from the server. It does not modify any data.
Example: Getting user details.
2. **POST** – Used to send data to the server to create a new resource.
Example: Creating a new user account.
3. **PUT** – Used to update an existing resource completely.
4. **DELETE** – Used to remove a resource from the server.
5. **PATCH** – Used to partially update a resource.

Que : HTTP Status Codes

HTTP status codes are standard response codes returned by the server to indicate the result of a client's request.

They are divided into five categories:

1 1xx – Informational

Indicates the request was received and is being processed.

2 2xx – Success

Indicates the request was successfully processed.

Example:

- **200 OK** – Request successful
- **201 Created** – Resource created successfully

3 3xx – Redirection

Indicates further action is needed.

Example:

- **301 Moved Permanently**

4 4xx – Client Error

Indicates an error from the client side.

Example:

- **400 Bad Request**
- **401 Unauthorized**
- **404 Not Found**

5 5xx – Server Error

Indicates an error on the server side.

Example:

- **500 Internal Server Error**

HTTP status codes help the client understand whether the request was successful or failed.

 Fill in the Blanks – Node.js :

1. Node.js is built on the _____ JavaScript engine.
2. Node.js is primarily used for building _____ side applications.
3. The default package manager for Node.js is _____.
4. The file that contains project metadata is called _____.
5. Node.js uses an _____-driven architecture.
6. Node.js follows a _____ I/O model.
7. To import a module in Node.js, we use the _____ keyword.
8. The global object in Node.js is called _____.
9. The built-in module used to create a server is _____.
10. The method used to start a server is _____.
11. The folder where installed packages are stored is _____.
12. The file that locks exact dependency versions is _____.
13. Node.js runs on the _____ engine.
14. The module used to work with file systems is called _____.
15. To install a package, we use the command _____.
16. The method used to read a file asynchronously is _____.
17. Node.js is _____-threaded by default.
18. The object that contains request data in HTTP server is _____.

19. The object used to send response back to client is _____.
20. The event-driven model in Node.js is handled by the _____ loop.
21. The built-in module used to handle events is _____.
22. To export a module, we use _____.
23. The core module used for working with paths is _____.
24. Node.js supports _____ programming style.
25. The command to initialize a new Node project is _____.
26. Environment variables are stored in a file named _____.
27. The method used to write data to a file asynchronously is _____.
28. Node.js applications can handle multiple requests due to its _____ nature.
29. The built-in module used to work with operating system details is _____.
30. The method used to end an HTTP response is _____.